

CS4263 Distributed Systems

2021 Batch Semester 8 Programming Assignment

Project Report

Team Members:

210372D - K.C.K.Manawathilake

210373G - S.M.A.N.A.Manchanayake

Assumptions of the System

- Nodes are assumed to know their own energy levels.
- Any delays in communication and clocks are assumed to be negligible compared to the timesteps of message processing.
- Messages are communicated as broadcast messages, any node at any distance from the sender can intercept and process a message. The energy consumption of all messages is equal (2 units of energy).
- All nodes have a unique ID value.
- Since the heartbeat requirement is not specified in the assignment, a 4-time-unit interval heartbeat is used, broadcasted to all members of the respective group by the leader.
- The time steps of the process per node are large enough compared to the system to broadcast any intercepted messages within a time interval.

System Initiation

Initially, there are no leaders in the system and each node does not know the existence of other nodes. Initially, ID values are allocated to the nodes to identify each of them uniquely.

The following are the system initiation steps:

1. All members broadcast a "Self Introduction" message denoting their ID, location, and energy level. This is somewhat similar to indication of election participation, but since it is the initiation process, it happens across the entire system.
2. The node with the highest energy level (or, if energy levels are equal, the node with the lowest ID) starts acting as the leader, runs the clustering algorithm, and broadcasts the information of all groups and their leaders.
3. When nodes receive that message, they thereafter respond only to their respective leaders.

The system initiation process ends here.

Clustering Strategy

The main objective of the clustering algorithm is *to increase the lifespan of the system*. To achieve this, the system should reduce the number of transmissions (and leader elections). This is done by selecting a leader with a long lifespan (higher energy level) and minimizing the number of groups (thereby reducing the number of leaders and heartbeat transmissions).

Clusters are created based on available energy levels. From a group of nodes, the node with the highest energy is selected as the leader. It then identifies all nodes within a radius of 20 units around it. From the remaining nodes, the node with the highest energy is selected, and its child nodes are identified. This process is repeated until no free nodes remain.

Leader Election Strategy

A node is elected as the leader if it has the maximum energy compared to other nodes. If multiple nodes have the same maximum energy, the node with the minimum ID is selected to break the tie. A leader sends a heartbeat every 4 time steps, and each node expects to receive a heartbeat at least once within a 5-time-step interval. If no heartbeat is received from the leader within that period, every node that detects the leader failure sends a “Leader Failure” message along with its energy level. This indicates that the node is participating in the election of a new leader. The leader is elected using a bully algorithm. If a node has more energy than all received candidates, it broadcasts itself as the new leader.

In the case that the leader dies before its children, and the newly elected leader cannot serve all nodes due to being out of reach of the 20-unit Euclidean distance constraint, it reclusters the group using the algorithm described above and transmits a “Group Allocation” message to all nodes. The nodes then receive that message and reconstruct the groups as specified.

Challenges

1. Identifying a suitable clustering strategy

A suitable clustering strategy needed to be selected and following are a few challenges that we faced when identifying that

- Creating unnecessary clusters increased the number of leaders, increasing the number of heartbeat transmission intern reduce the final life optimization target.
- Clusters are limited to a 20 radius limit from the leader.
- The clustering algorithm probably needs to be simple considering that these are implemented in the node devices itself.
- Incorrect clustering would create increased number of clusters in the case of a leader death

It was hard to find a perfect solution to address all these issues. The above mentioned clustering algorithm was selected as it specifically enforces the 20 radius rule, simple in complexity and uses a greedy approach by taking the energy levels into consideration to avoid the number of new leader elections

2. Identifying a suitable leader election strategy

A bully algorithm was used, paired with the above-mentioned greedy clustering algorithm and node-level buffers, to minimize the number of transmissions. Nodes transmit a “Leader Failure” message as soon as they detect that the leader is missing. The nodes also maintain a buffer and take actions only at fixed time intervals, while considering other available messages in the buffer.

This approach increases the probability that nodes within a group receive all “Leader Failure” messages, reducing repeated transmissions to other nodes during leader election. This minimizes the total number of transmissions and thereby increases the lifespan of the system.

3. Identifying a suitable heartbeat protocol

The heartbeat interval was not specified in the assignment. Choosing a suitable heartbeat protocol was necessary to optimize system lifespan. A frequent heartbeat would be impractical and energy consuming, while a large interval between heartbeats would result in longer periods where nodes remain unaware of a missing leader. A balanced value of 4 was selected to represent a midpoint between these two extremes. A heartbeat absence threshold of 5 time steps was defined for each node to account for potential communication inconsistencies.

If there are no other members in a group apart from the leader, sending a heartbeat is unnecessary, as there are no recipients. However, since the assignment does not define detailed energy consumption models for nodes, applying this optimization would cause the system to converge toward all nodes acting as leaders for maximum lifespan efficiency. Therefore, this optimization was not adopted.

Visualization and Rendering Strategy

The visualization engine is designed to provide immediate feedback without disrupting the simulation's background processing. To achieve this, **Matplotlib** is launched in interactive mode allowing both rendering and updating dynamically in the background. We made the visualizer so that it utilizes short UI pauses (`plt.pause(0.5)`) to clamp the simulation to a human-readable speed. Nodes are rendered dynamically as a scatter plot where each node possesses a unique ID and all members belonging to a specific group are assigned the same color.

Each timestep stored in the system's timeline dictionary is captured as a deep copy of the network state. So, manipulating the timeline slider never alters the historical integrity of the nodes. If the selected node is a Leader, the visualizer dynamically overlays a dashed circle representing its exact Euclidean coverage radius 20units. The user can replay to auto-step through the historical timeline at their own pace.

*** For creating the simulation visualizations, we used AI assistance to produce clean and visually appealing renderings of the results.*